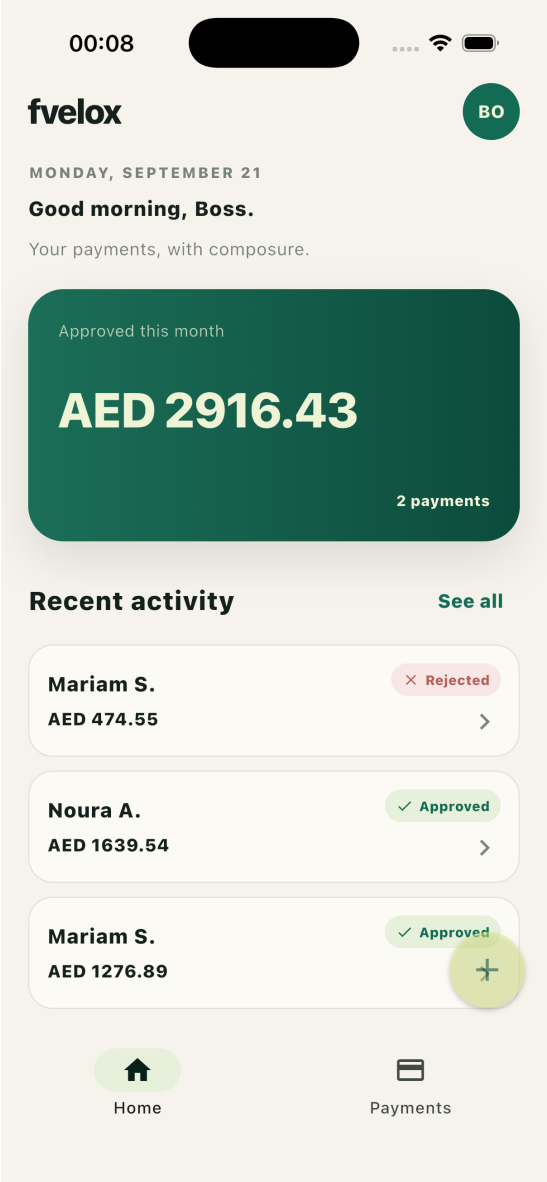
Fvelox

Payment Approval Flow

Technical challenge handoff

A complete guide to the implemented flow, source code, test coverage and next steps.

APP PREVIEW



1. Delivery

Item	Link / file
Source code	https://github.com/nqhhdev/fvelox
Documentation	README.md and docs/ are included in the source repository.
Demo	A live demo or short recording can be provided during review.

2. Scope delivered

The implementation covers the Payment Approval Flow challenge as a small production-minded feature:

- Splash screen, light theme and bottom navigation for Home and Payments.
- Home monthly summary and recent decided activity.
- Payments list ordered newest first with party, AED amount and status.
- Payment Details for approved and rejected payments only.
- Global draggable Debug FAB that creates an incoming pending request.
- Approval overlay shown over the current screen with masked party and amount.
- Approve and Reject actions protected by demo OTP 8888.
- Consistent state update across Home, Payments and Payment Details.
- Swipe-to-delete for decided payments with refreshed state.
- Loading, empty, error, offline indicator and retry-oriented UI states.

3. How to try the app

1. Install and open the release APK. No backend credentials are required for this deterministic demo.

2. Wait for the splash screen. Home shows the current month summary and recent activity.

3. Open Payments from the bottom navigation and review decided payments.

4. Tap an approved or rejected row to see full details, then use Back to return to the originating screen.

5. Tap the floating Debug FAB. A new incoming request is created and shown as an overlay over the current screen.

6. Verify that the other party and amount are masked while the reference is visible in full.

7. Choose Approve or Reject. Enter demo OTP 8888 and tap Confirm. All amounts use AED.

8. To verify validation, enter 1234 first. The error appears directly below the OTP field.

9. After approval, the payment appears at the top of Payments and the Home summary updates.

10. After rejection, the payment remains visible in the list but does not increase the Home total or count.

11. Swipe a decided payment from right to left to delete it and observe the list and summary refresh.

4. Acceptance criteria status

Area	Status and evidence
Home	Implemented and covered by native integration tests: monthly approved summary, recent activity, navigation and rejected payments excluded from totals.
Payments list	Implemented and covered: all decided payments, newest-first ordering, recognizable rows and new decisions inserted at the top.
Payment Details	Implemented and covered: approved/rejected details only, pending remains overlay-only, and back returns to the originating screen.
Debug FAB	Implemented and covered: global visibility, drag behavior, session position retention and creation of incoming requests.
Approval	Implemented and covered: overlay, masked data, full reference, OTP validation, approve/reject navigation, toasts and shared state refresh.
Delete	Implemented and covered: swipe delete, backend-first mutation, immediate row removal after success and refreshed summary/list.

5. Product behavior and decisions

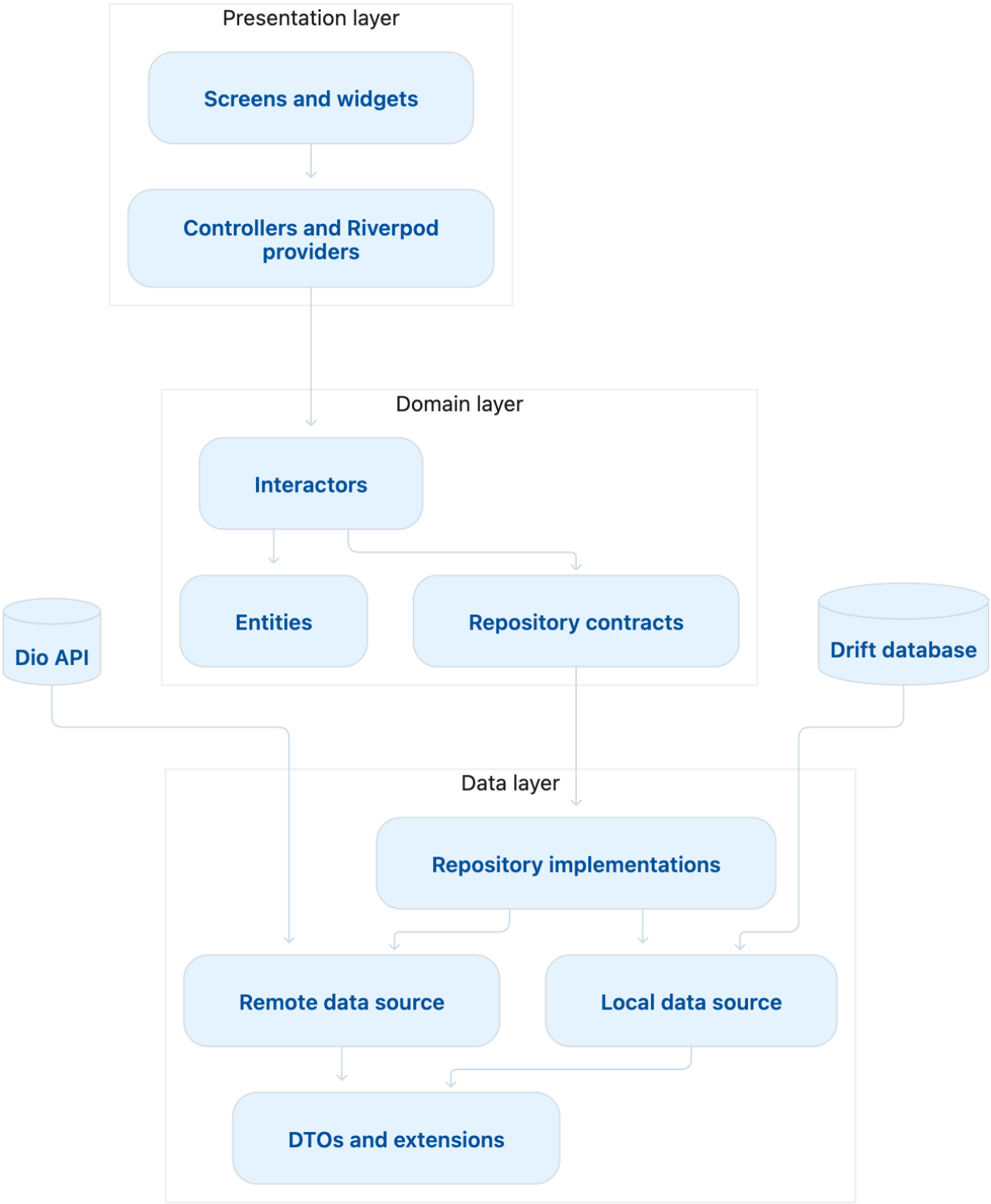
• AED is the only displayed currency for this challenge, formatted consistently to two decimal places.
• Pending requests exist only in the approval overlay and are never written to the decided-payment cache.
• Only approved and rejected payments can open Payment Details.
• Rejected payments remain visible in Payments and Recent activity but do not count as money moved.
• The demo OTP is hardcoded as 8888. There is no retry counter or lockout in this challenge version.
• Approve and Reject are not optimistic. The final state is shown only after the repository confirms the mutation.
• After decide or delete, controllers re-read authoritative repository data so Home, Payments and Details remain consistent.
• The Debug FAB is an explicit demo input for creating an incoming request; it is not a production notification channel.

Architecture

The full architecture diagram is provided on the following page.

6. Architecture

Feature-first Clean Architecture with Riverpod, Dio and Drift:



Runtime flow: UI -> Riverpod controller -> domain interactor -> repository contract -> remote or local data source. The repository is the UI-facing source of truth.

7. Technology baseline

Area	Choice
Toolchain	Flutter 3.38.9 stable, Dart 3.10.8, Android SDK 36.0.0, Xcode 26.2, CocoaPods 1.16.2, Java 17 for Gradle.
State and navigation	flutter_riverpod 2.6.1, riverpod 2.6.1, go_router 17.5.0. Manual providers are used instead of Riverpod code generation.
Network	dio 5.11.1, connectivity_plus 7.3.1, typed AppFailure mapping and debug-only API tracing.
Persistence	drift 2.34.4 and drift_flutter 0.3.1 for structured local data and cache synchronization.
UI	Light Material/Cupertino design system, flutter_floaty 0.2.0 for the draggable FAB, pinput 6.0.2 for OTP input and intl 0.20.2.
Testing	flutter_test, native integration_test, Patrol 4.10.0, build_runner and JSON/Drift generators.

8. Source setup

Required environment: Flutter stable 3.38.9, Dart 3.10.8 and a connected Android or iOS device/simulator.

Install dependencies and run the app:

```
cd fvelox
flutter doctor -v
flutter pub get
flutter run
```

Generate Drift and JSON code after changing tables, DAOs or annotated models:

```
dart run build_runner build --delete-conflicting-outputs
```

Localization files live under assets/l10n/ and use Flutter's generate: true configuration.

9. Testing

Suite	Command and scope
Unit/widget	flutter analyze and flutter test. Covers domain rules, repositories, controllers, key UI states, errors, offline widgets and persistence behavior.
Native acceptance	flutter test integration_test/acceptance_criteria_test.dart -d <device-id>. Covers the business flow with deterministic DemoPaymentRepository data.
Patrol acceptance	patrol test --target integration_test/app_launch_test.dart --dart-define=FVELOX_PATROL_TEST=true. Covers device launch and the Patrol payment acceptance path.

Current verification recorded in the source docs:

[x] flutter analyze passes.
[x] flutter test passes.
[x] Native acceptance flow is verified on the configured Android API 36 emulator and iOS simulator.
[x] Release APK builds successfully for Android arm64.
[x] Patrol test bundle builds successfully.

The remaining device-level release checks are environment checks: free Android emulator storage, resolve the local iOS XCTest runner issue, install the release APK on a clean device, and perform a manual airplane-mode check.

10. Build and handoff

Debug APK:

```
flutter build apk --debug
```

Release APK:

```
flutter build apk --release

Output: build/app/outputs/flutter-apk/app-release.apk
```

If the local Flutter configuration points Gradle to a different JDK, use Java 17 for the Android build:

```
flutter config --jdk-dir=<path-to-jdk-17>
```

11. Offline behavior

Offline is not part of the primary reviewer demo flow. The app already has the boundaries for read-only cached behavior, but full device verification is a release check and can be expanded if more time is available.

- Widget coverage exists for the offline banner.
- Repository coverage blocks create, decide and delete while offline.
- Reads can fall back to Drift cached data.
- Financial mutations are never queued or replayed offline.
- A full airplane-mode device check remains optional for this handoff.

12. Limitations and future work

The implementation is intentionally scoped for the two-day technical challenge. A production fintech version could add:

- Server-validated OTP, biometric verification, expiry, retry limits, lockout and risk-based step-up authentication.
- Secure authentication/token storage and encrypted local persistence.
- Payment audit events, idempotency, stronger reconciliation and backend contract documentation.
- Push notifications for new payment requests.
- Pagination, refresh strategy, reconnect handling and broader observability/monitoring.
- CI/CD, signed distribution, golden tests, low-end performance profiling and wider device coverage.
- Full offline read-only UX and manual device verification; no offline financial mutation queue.

13. Reviewer note

The implementation aims to show clear boundaries, careful financial state transitions and a practical test strategy while keeping the challenge scope focused. I would be happy to walk through the code in a live demo and welcome code review or technical feedback from the team, especially around fintech domain modeling, security and production mobile engineering practices.